



南开大学
Nankai University

《软件测试与维护》大作业报告

Online-Boutique 运维实验与 ISSRE24-KPIRoot 论文复现

实验标题： Online-Boutique 运维实验与 ISSRE24-KPIRoot 论文复现

课程名称： 软件测试与维护

报告范围： 组内两篇论文复现任务中的一篇：ISSRE24-KPIRoot

微服务系统： JoinFyc/Online-Boutique

参与者： 王新杰（学号：2311901）；王子祺（学号：2312385）

GitHub： [https://github.com/Sinclair987/
software-testing-final-project](https://github.com/Sinclair987/software-testing-final-project)

指导教师： 张圣林

完成日期： 2026-06-06

说明：课程大作业以小组形式完成，基本要求包含两篇论文复现。本报告仅记录组内其中一篇论文 ISSRE24-KPIRoot 的复现工作；另一篇论文复现与智能体封装由组内其他成员另行整合。

目录

摘要	3
1 作业范围与系统概况	3
2 阶段一：微服务系统部署	4
3 阶段二：监控、故障注入与数据采集	4
4 阶段三：Selenium 与 JMeter 测试	6
5 ISSRE24-KPIRoot 论文方法	7
6 阶段四：KPIRoot 复现实验结果	8
7 差异、局限与优化方向	10
8 参与者与贡献	10
9 总结	11
参考资料	11

摘要

本报告围绕 JoinFyc/Online-Boutique 开源微服务系统，记录本地部署、监控维护、故障注入、黑盒测试以及 ISSRE 2024 论文 KPIRoot 的复现工作。实验使用 Minikube 部署 Online-Boutique，使用 Prometheus、Grafana、Blackbox Exporter 与 ChaosMesh 采集正常与故障状态下的时序 KPI 数据，使用 Selenium 和 JMeter 验证系统功能与性能，最后基于采集数据实现 KPIRoot 中的 PAA、SAX、相似度分析、Granger 风格因果得分和综合排序。实验结果显示，在 paymentservice CPU Stress、frontend CPU Stress 和 paymentservice Pod Kill 三组故障数据中，复现算法均能将真实根因服务排在第一位。

关键词：Online-Boutique；Prometheus；Grafana；ChaosMesh；Selenium；JMeter；KPIRoot；根因定位

1 作业范围与系统概况

课程要求围绕微服务系统完成部署、测试、维护和论文复现，并在报告中展示微服务系统、Selenium/JMeter 测试、ChaosMesh 故障注入、Prometheus/Grafana 监控、异常数据采集与论文算法效果。本项目选择第二档要求中的 Online-Boutique，而不是实验指南中的 SockShop，因此系统规模和服务组成更接近真实云原生应用。

需要特别说明的是，课程要求小组整体复现两篇异常检测或故障诊断相关论文。本报告对应组内其中一篇，即 ISSRE24-KPIRoot。报告中的代码、数据和结果均来自公开仓库：

<https://github.com/Sinclair987/software-testing-final-project>

项目	内容
微服务系统	JoinFyc/Online-Boutique
本地集群	Minikube
命名空间	online-boutique
监控组件	Prometheus、Grafana、Blackbox Exporter
故障注入	ChaosMesh
测试工具	Selenium、JMeter
复现论文	ISSRE 2024 - KPIRoot: Efficient Monitoring Metric-based Root Cause Localization in Large-scale Cloud Systems

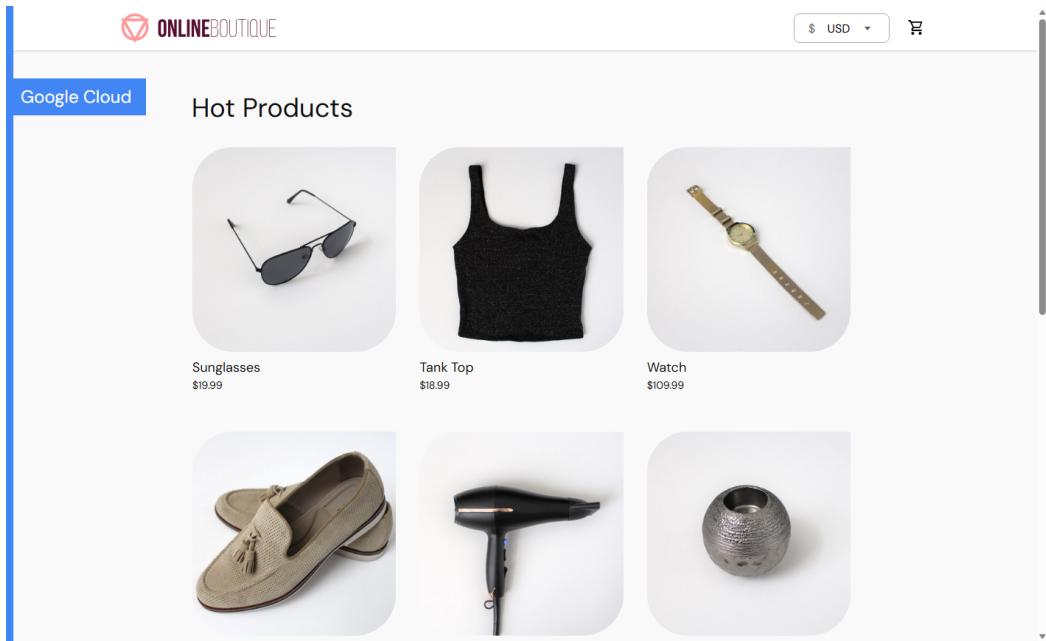


图1 Online-Boutique 前端首页

2 阶段一：微服务系统部署

阶段一完成 Online-Boutique 在 Minikube 中的部署。部署时使用 release/kubernetes-manifests.yaml 中的预构建镜像，并创建 online-boutique 命名空间。部署完成后，12 个业务 Pod 均进入 1/1 Running 状态，frontend 服务通过本地端口转发暴露到 127.0.0.1:8088。

```
kubectl apply -n online-boutique -f .\FinalProject\Online-Boutique\release\kubernetes-manifests.yaml
kubectl wait --for=condition=available deployment --all -n online-boutique --timeout=300s
kubectl port-forward -n online-boutique service/frontend 8088:80
```

直接访问 Minikube NodePort 在 Windows 环境下不够稳定，因此实验中采用 kubectl port-forward 作为稳定前端入口。项目保留 resume-online-boutique.ps1 和 port-forward-frontend.ps1 作为重启后恢复脚本。

3 阶段二：监控、故障注入与数据采集

阶段二复用 monitoring 命名空间中的 Prometheus 与 Grafana，并部署 Blackbox Exporter 对 Online-Boutique 前端进行探测。Prometheus 能采集容器 CPU、内存、文件系统读写、Pod 状态、容器重启次数以及前端 probe_success/probe_duration_seconds 等指标。Grafana Dashboard 用于观察故障前、故障中和恢复后的指标变化。

组件	作用
Prometheus	采集 Kubernetes、cadvisor、kube-state-metrics、node-exporter 与 Blackbox 指标
Grafana	展示 Online-Boutique Dashboard，包括 CPU、内存、运行状态和探针指标
Blackbox Exporter	提供前端可用性和响应耗时指标，作为服务质量告警 KPI
ChaosMesh	注入 CPU Stress 和 Pod Kill 故障

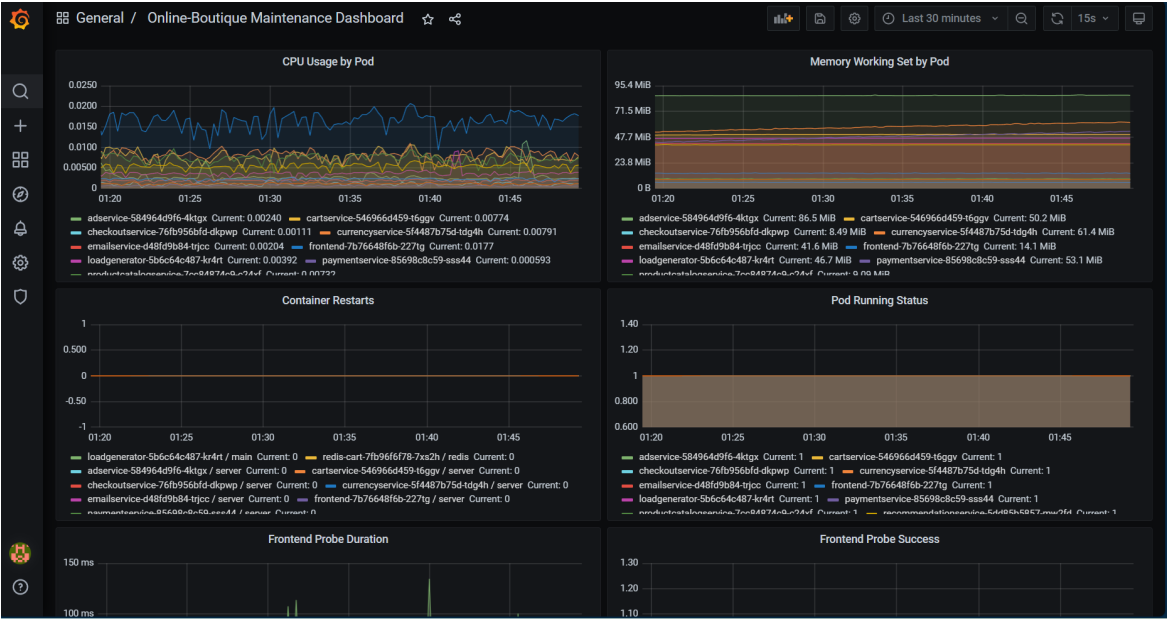


图2 Grafana 正常基线监控面板

阶段二最终保留三组故障数据。每组数据均包含 metadata.yaml、Prometheus 原始 query_range 导出、处理后的 kpi_matrix.csv、series_labels.json 和截图证据。

数据集	故障类型	目标服务	矩阵规模	阶段四真实根因
stress-paymentservice-cpu-001	CPU Stress	paymentservice	115 行 × 62 列	cpu__paymentservice
pod-kill-paymentservice-001	Pod Kill	paymentservice	67 行 × 62 列	paymentservice 服务
stress-frontend-cpu-001	CPU Stress	frontend	68 行 × 60 列	cpu__frontend

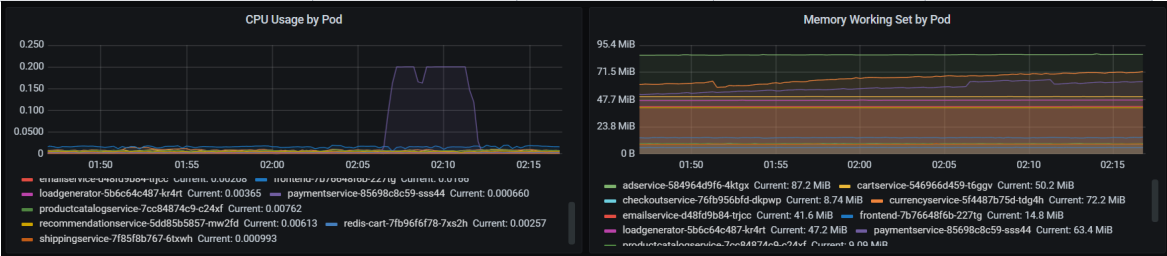


图3 paymentservice CPU Stress 故障期间 CPU 指标升高

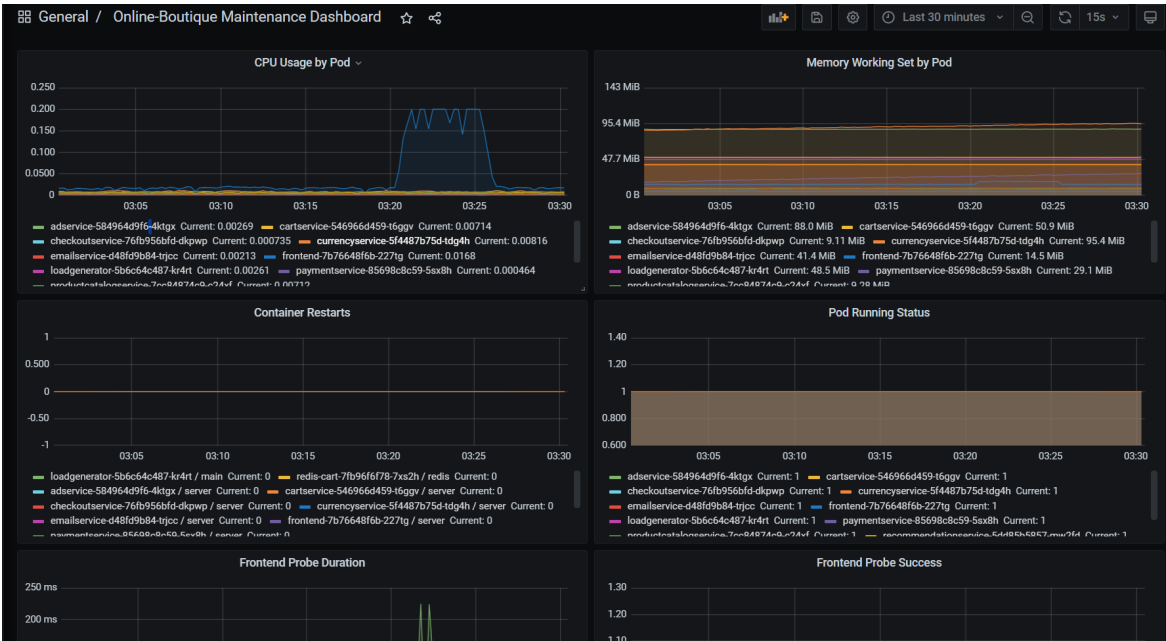


图4 frontend CPU Stress 故障期间 CPU 指标升高

paymentservice CPU Stress 场景中，cpu__paymentservice 从约 0.00064 升至故障窗口平均约 0.16999；frontend CPU Stress 场景中，cpu__frontend 从约 0.016 升至故障窗口平均约 0.170，且前端探针耗时出现升高。Pod Kill 场景中 Kubernetes 快速创建替换 Pod，整体指标变化较弱，但原始 Prometheus 数据保留了旧 Pod 与新 Pod 的身份变化。

4 阶段三：Selenium 与 JMeter 测试

阶段三使用 Selenium 对 Online-Boutique 前端核心购物流程进行功能测试，使用 JMeter 对首页、商品详情、加入购物车、购物车和结算流程进行性能测试。Selenium 运行环境为 Microsoft Edge，JMeter 版本为 5.1.1。

测试用例	指标	类型	时间 ms
test_home_page_lists_products	home_page_load_ms	page_load	1803.1
test_product_detail_page	product_detail_page_load_ms	page_load	1750.6
test_add_to_cart	add_to_cart_interaction_ms	interaction	165.09
test_checkout_flow	checkout_submit_interaction_ms	interaction	228.88

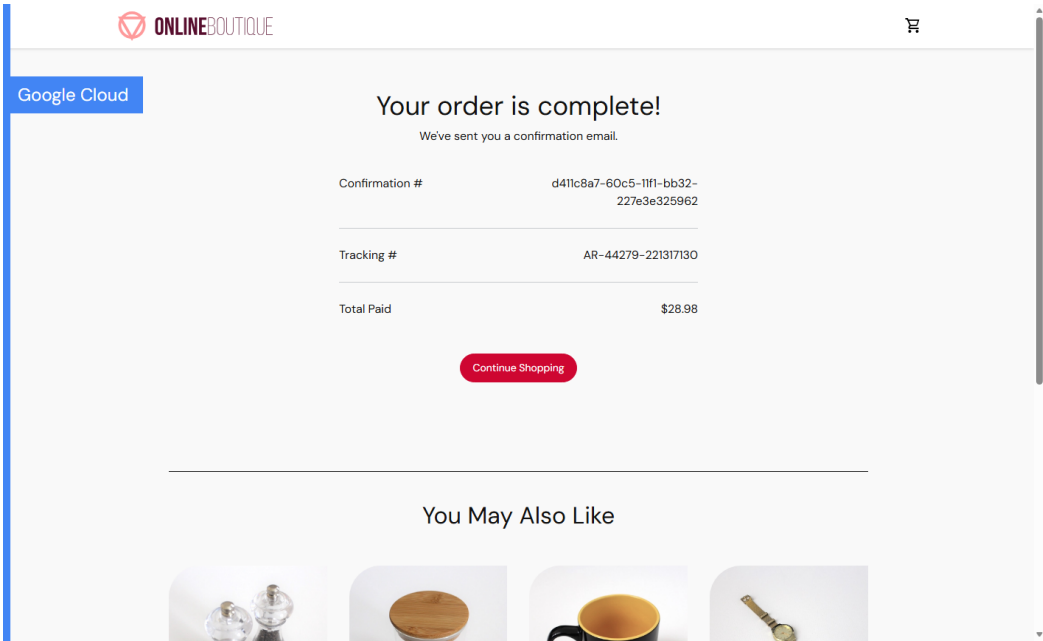


图5 Selenium 结算流程完成页面

Run	Main Samples	Raw Samples	Errors	Error %	Avg ms	P90 ms	P95 ms
smoke-001	5	7	0	0.0	31.8	49.8	51.4
normal-001	250	350	0	0.0	35.83	77.0	83.0
higher-001	500	700	0	0.0	33.53	73.0	78.0

Statistics												
Requests		Executions			Response Times (ms)					Throughput	Network (KB/sec)	
Label	#Samples	KO	Error %	Average	Min	Max	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	700	0	0.00%	29.20	4	145	70.90	77.00	88.00	23.93	253.82	6.68
01 Home Page	100	0	0.00%	43.45	20	145	77.00	81.85	144.46	3.43	36.08	0.40
02 Product Detail	100	0	0.00%	26.28	9	95	59.00	67.00	94.99	3.44	27.37	0.67
03 Add To Cart	100	0	0.00%	36.88	14	91	78.00	82.95	90.97	3.44	56.24	1.63
03 Add To Cart-0	100	0	0.00%	8.24	4	79	15.20	41.85	78.76	3.44	0.32	1.01
03 Add To Cart-1	100	0	0.00%	28.48	9	79	70.90	75.90	78.99	3.44	55.94	0.62
04 View Cart	100	0	0.00%	33.62	9	88	73.00	76.95	87.89	3.44	55.88	0.62
05 Checkout	100	0	0.00%	27.43	14	92	65.50	73.90	91.96	3.44	23.41	1.79

Errors			
Type of error	Number of errors	% in errors	% in all samples

Top 5 Errors by sampler			
-------------------------	--	--	--

图6 JMeter higher-001 统计结果与错误率

Selenium 共 4 个用例全部通过，页面加载时间约 1.75 到 1.80 秒，关键交互响应时间低于 0.25 秒。JMeter 三组负载均未出现错误，higher-001 中 500 个主采样器的平均响应时间约 33.53 ms，P95 响应时间约 78 ms，说明在本地实验负载下系统保持稳定。

5 ISSRE24-KPIRoot 论文方法

KPIRoot 的目标是在系统级 alarm KPI 出现异常后，从大量底层 KPI 中定位最可能导致异常的根因 KPI。原论文面向 Cloud H 大规模云系统，输入包括主机集群层面的 alarm KPI 和 VM 级别的 CPU、内存、I/O、带宽等 KPI。其核心思想是：真正的根因 KPI 在异常窗口内应当与 alarm KPI 有相似的异常形状，并且其变化应当在时序上能够解释 alarm KPI 的变化。

原论文概念	本项目映射
Host/cluster alarm KPI	synthetic_total_cpu、synthetic_total_memory、前端探针耗时与成功率
VM KPI	Online-Boutique 各服务的 CPU、内存、文件系统、运行状态等 KPI
Root-cause VM	被 ChaosMesh 注入故障的目标服务

复现实现包含以下步骤：读取阶段二 kpi_matrix.csv，构造系统级告警 KPI，缺失值处理与标准化，使用 PAA 降维，使用 SAX 将连续序列离散化为符号序列，根据故障注入元数据选择异常窗口，计算 SAX-Jaccard 相似度，计算 Granger 风格因果得分，最后按 $0.9 \times \text{similarity} + 0.1 \times \text{normalized_causality}$ 生成综合得分并排序。

```
src/kpiroot/algorithm.py
src/kpiroot/data.py
src/kpiroot/cli.py
tests/kpiroot/test_algorithm.py
```

6 阶段四：KPIRoot 复现实验结果

故障场景	告警 KPI	真实根因服务	Top-1 KPI	根因服务排名	Hit@1	Hit@3	Hit@5
pod-kill-paymentservice-001	synthetic_total_memory	paymentservice	memory__paymentser vice	1	True	True	True
stress-frontend-cpu-001	synthetic_total_cpu	frontend	cpu__frontend	1	True	True	True
stress-paymentservice-cpu-001	synthetic_total_cpu	paymentservice	cpu__paymentservice	1	True	True	True

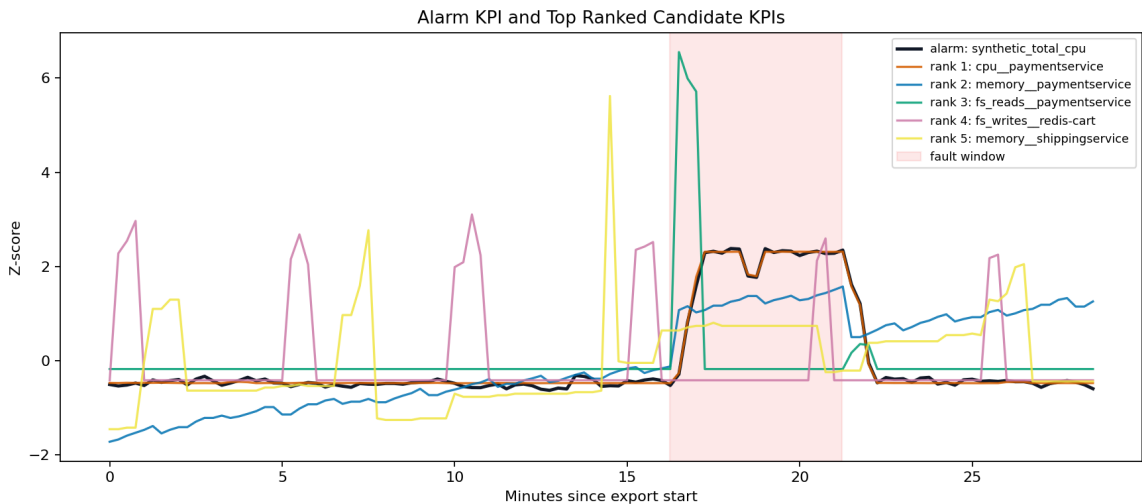


图7 paymentservice CPU Stress：告警 KPI 与 Top 候选 KPI 对比

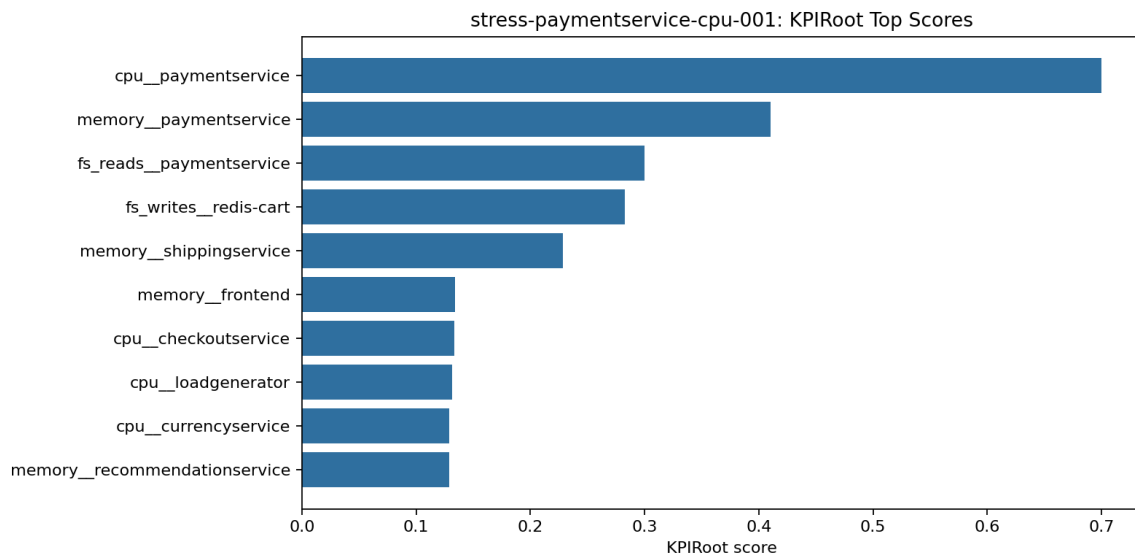


图8 paymentservice CPU Stress: Top-K 根因得分

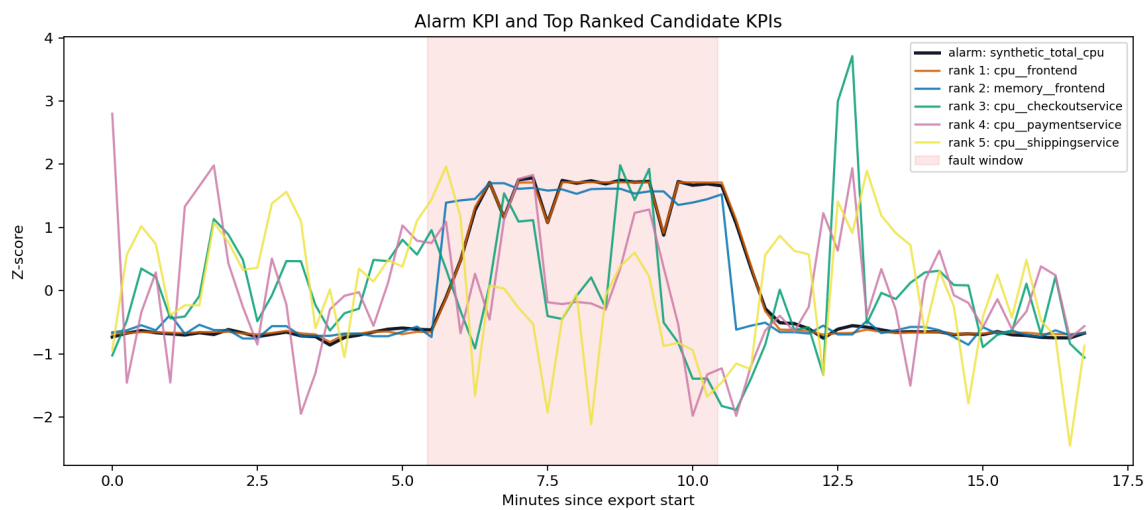


图9 frontend CPU Stress: 告警 KPI 与 Top 候选 KPI 对比

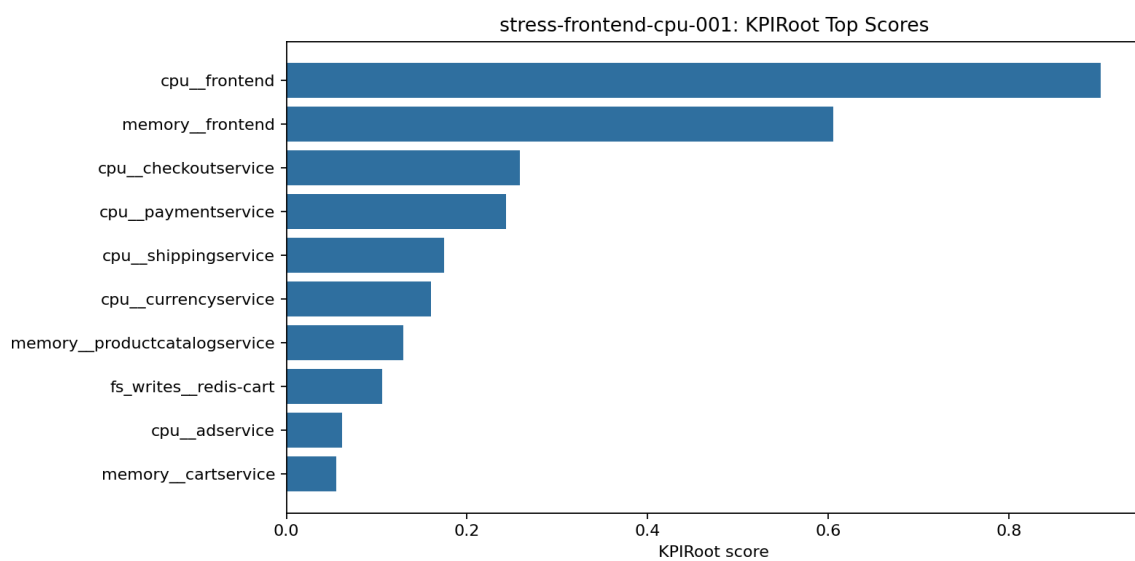


图10 frontend CPU Stress: Top-K 根因得分

三组故障中，综合 KPIRoot 排序均将真实根因服务排在第一位。其中两组 CPU Stress 的可解释性最强：系统级 synthetic_total_cpu 升高后，算法分别定位到 cpu__paymentservice 和 cpu__frontend。Pod Kill 场景中，服务级矩阵弱化了单个 Pod 身份变化，算法仍将 paymentservice 相关内存 KPI 排在第一，可作为边界案例。

故障场景	方法	Top-1 KPI	根因服务排名	Hit@1	Hit@3	Hit@5
pod-kill-paymentservice-001	similarity_only	memory__paymentservice	1	True	True	True
pod-kill-paymentservice-001	causality_only	fs_writes__redis-cart	11	False	False	False
pod-kill-paymentservice-001	kpiroot_combined	memory__paymentservice	1	True	True	True
stress-frontend-cpu-001	similarity_only	cpu__frontend	1	True	True	True
stress-frontend-cpu-001	causality_only	cpu__checkoutservice	9	False	False	False
stress-frontend-cpu-001	kpiroot_combined	cpu__frontend	1	True	True	True
stress-paymentservice-cpu-001	similarity_only	cpu__paymentservice	1	True	True	True
stress-paymentservice-cpu-001	causality_only	memory__shippingservice	9	False	False	False
stress-paymentservice-cpu-001	kpiroot_combined	cpu__paymentservice	1	True	True	True

消融实验显示，在本项目较短的课程实验时序数据上，单独使用 Granger 风格因果得分并不稳定；仅使用 SAX-Jaccard 相似度和使用 KPIRoot 综合得分均能稳定命中真实根因。该现象与原论文中“相似度和因果分析均有贡献”的结论并不矛盾，因为原论文使用的是更长、更复杂的工业数据，而本项目故障窗口较短、故障类型也更直接。

7 差异、局限与优化方向

- 原论文使用 Cloud H 工业环境的大规模主机集群/VM KPI，本项目使用 Online-Boutique 微服务级 KPI，因此数据规模和实体层级不同。
- 原论文的 alarm KPI 来自真实云系统告警，本项目使用聚合 CPU、聚合内存和前端探针指标模拟系统级告警。
- 原论文自动检测异常段，本项目优先使用 ChaosMesh 故障注入时间窗口，自动趋势检测作为备用逻辑。
- Pod Kill 的 Pod 身份变化在服务级宽表中被部分合并，后续可以保留 Pod 级 KPI 或引入事件日志，以提升该类故障的可解释性。
- 后续若扩展为完整小组报告，需要合并另一篇论文复现结果，并补充智能体封装部分的设计和效果。

8 参与者与贡献

参与者	学号	本报告对应工作
王新杰	2311901	参与 ISSRE24-KPIRoot 阅读、环境部署、监控采集、故障数据整理、算法复现与报告整理。
王子祺	2312385	参与 ISSRE24-KPIRoot 阅读、复现方案讨论、结果校核与报告整理。

9 总结

本报告完成了 Online-Boutique 微服务系统部署、Prometheus/Grafana 监控、ChaosMesh 故障注入、Selenium/JMeter 测试以及 ISSRE24-KPIRoot 根因定位算法复现。实验结果表明，经过阶段二采集和处理的 KPI 数据能够支撑 KPIRoot 的核心流程，复现实验可以在三组故障场景中将真实根因服务排在第一位。与原论文相比，本项目的数据规模较小、场景经过课程实验简化，但已经覆盖了 PAA、SAX、相似度分析、因果得分、综合排序和消融实验等关键环节，能够作为组内一篇论文复现成果提交与展示。

参考资料

- Wenwei Gu et al. KPIRoot: Efficient Monitoring Metric-based Root Cause Localization in Large-scale Cloud Systems. ISSRE 2024.
- JoinFyc/Online-Boutique: <https://github.com/JoinFyc/Online-Boutique>
- 项目仓库: <https://github.com/Sinclair987/software-testing-final-project>
- 软件测试与维护（2026年春）大作业要求。